

Contents

Part A: Project Initialization and Tool Setup	2
A1: Java Project Selection	2
A2: Tool Installation and Verification	3
Part B: Code Smell Analysis and Refactoring	7
B1: SonarQube Analysis and Metrics Extraction	7
B2: The Five Code Smell Categories - Deep Identification	10
B3: Smell Interaction and Prioritisation	15
B4: Refactoring Demonstration	16
Part C: Dependency, Coupling and Technical Debt	17
C1: Dependency Mapping	17
C2: Technical Debt Assessment	19
Part D: Dynamic Program Analysis	20
D1: Execution Trace	20
D2: Control Flow Graph (CFG)	23
D3: Abstract Syntax Tree Inspection	24
Part E: Data Smell Detection	25
E2: The 10 Data Smells Table	25
E3: Top 4 Smells by Risk	27
Part F: Schema Normalisation and Refactoring	29
F1: Normalisation of pat_master to 3NF	29
F1 Step 3: Before and After Comparison	31
F2: Five Schema Refactoring Scripts	31
F3: Refactoring Impact Summary Table	37
Part G: Data Migration Design and Execution	38
G2: Migration Plan Table	38
G3: ETL Transformation Script	39
G4: Post-Migration Validation	42

Part A: Project Initialization and Tool Setup

A1: Java Project Selection

What the system does?

The selected project is `HospitalManagement` by `rid17pawar`, available at <https://github.com/rid17pawar/HospitalManagement>. It is a web-based Hospital Management System built on the Java Spring MVC framework with Hibernate ORM for database persistence and Apache Maven as the build automation tool. The system manages core hospital operations including patient registration and record management, doctor profile management, appointment scheduling, and PDF prescription generation. The application follows a three-tier architecture: a presentation layer composed of Spring MVC controllers handling HTTP requests, a service layer encapsulating business logic, and a data access layer using Hibernate to communicate with a MySQL backend. The application is designed to be deployed on an Apache Tomcat servlet container and exposes its functionality through a JSP-based web interface.

Structure and technical profile:

The codebase is organised into separated packages for controllers, service interfaces and their implementations, data access objects, model entities, and web configuration. Key functional modules include patient management, doctor management, appointment handling, and billing support. The project uses standard Java EE annotations for Spring bean wiring and Hibernate mappings, and its Maven defines all dependencies including Spring MVC, Hibernate Core, MySQL Connector, and the JSTL tag library. One additional dependency `javax.servlet-api` version 3.1.0 was manually added to the `pom.xml` to resolve a servlet compilation conflict on the local environment, after which the project compiled successfully with `mvn compile -DskipTests`.

Why this project?

This project is an excellent candidate for code smell analysis. First, it was built as a practical demonstration project rather than as a production-grade system, which means it prioritises feature completeness over architectural cleanliness a pattern that reliably produces smells. Second, its Spring MVC controller classes are likely to exhibit Large Class and Long Method smells, as web controllers in tutorial-grade codebases often collapse multiple responsibilities input validation, business logic, and database coordination into single handler methods. Third, the parallel CRUD structure across Patient, Doctor, and Appointment modules creates strong conditions for Duplicate Code and Data Clumps.

Fourth, the direct interaction between controllers and DAOs in some paths bypasses the service layer, creating Feature Envy and Inappropriate Intimacy coupling smells.

A2: Tool Installation and Verification

```
C:\Users\FAIZAN>java -version
openjdk version "1.8.0_482"
OpenJDK Runtime Environment (Temurin)(build 1.8.0_482-b08)
OpenJDK 64-Bit Server VM (Temurin)(build 25.482-b08, mixed mode)

C:\Users\FAIZAN>mvn --version
Apache Maven 3.9.9 (8e8579a9e76f7d015ee5ec7bfcdc97d260186937)
Maven home: C:\apache-maven-3.9.9
Java version: 1.8.0_482, vendor: Temurin, runtime: C:\Program Files\Eclipse Adoptium\jdk-8.0.482.8-hotspot\jre
Default locale: en_US, platform encoding: Cp1252
OS name: "windows 11", version: "10.0", arch: "amd64", family: "windows"

C:\Users\FAIZAN>docker --version
Docker version 29.2.0, build 0b9d198

C:\Users\FAIZAN>sonar-scanner --version
18:41:38.270 INFO Scanner configuration file: C:\sonar-scanner\bin\..\conf\sonar-scanner.properties
18:41:38.276 INFO Project root configuration file: NONE
18:41:38.294 INFO SonarScanner CLI 8.0.1.6346
18:41:38.300 INFO Windows 11 10.0 amd64

C:\Users\FAIZAN>mysql --version
mysql from 11.4.3-MariaDB, client 15.2 for Win64 (AMD64), source revision 5ab81ffe0097a22a774957df28c5223cf0201de3

C:\Users\FAIZAN>python --version
Python 3.10.0

C:\Users\FAIZAN>pip --version
pip 21.2.3 from C:\Users\FAIZAN\AppData\Local\Programs\Python\Python310\lib\site-packages\pip (python 3.10)
```

Figure 1 All Required Tools

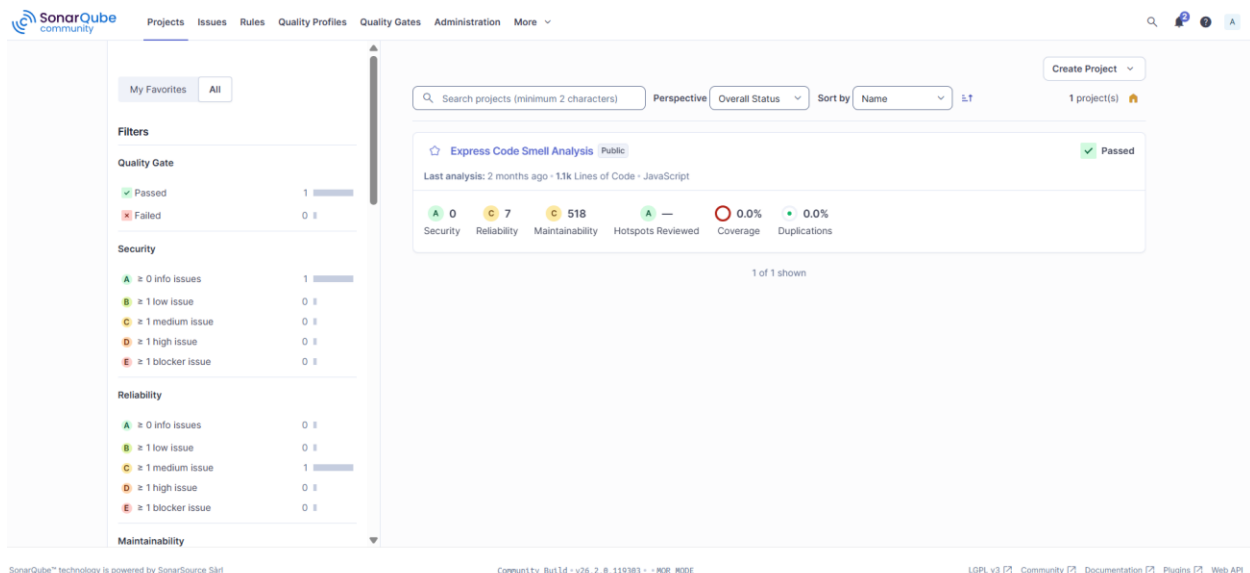


Figure 2 SonarQube Running Dashboard

```
C:\Users\FAIZAN>docker --version
Docker version 29.2.0, build 0b9d198

C:\Users\FAIZAN>sonar-scanner --version
17:33:16.151 INFO Scanner configuration file: C:\sonar-scanner\bin\..\conf\sonar-scanner.properties
17:33:16.159 INFO Project root configuration file: NONE
17:33:16.180 INFO SonarScanner CLI 8.0.1.6346
17:33:16.189 INFO Windows 11 10.0 amd64
```

Figure 3 Required Tools' Versions

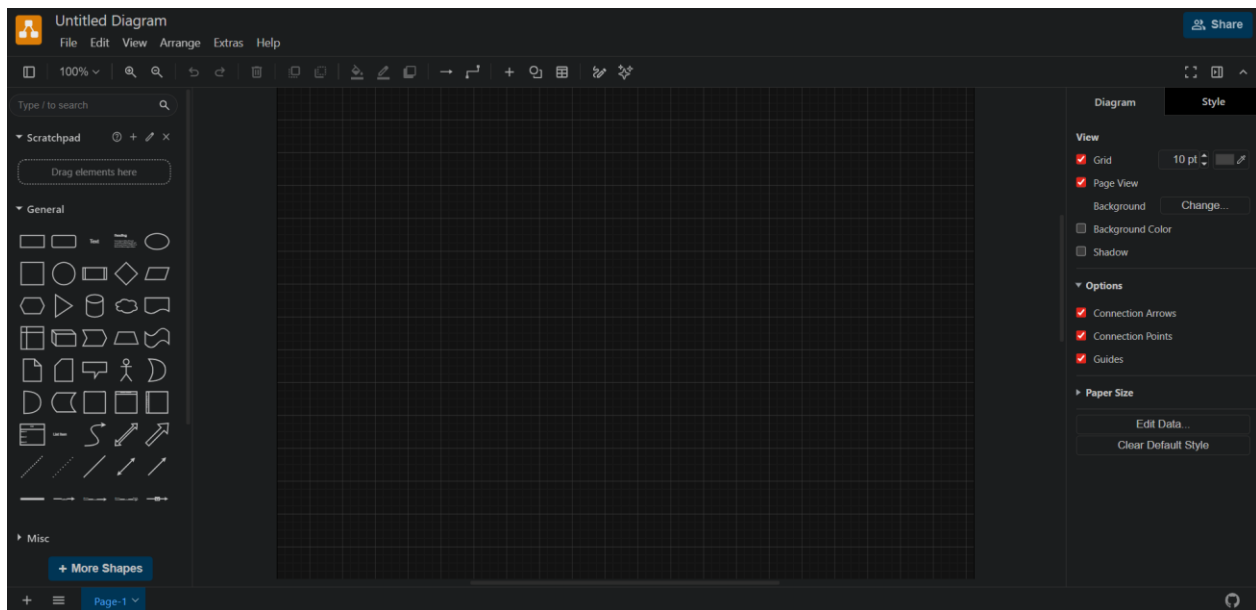


Figure 4 Draw io Opened on web

Python Tutor

Java Tutor

C Tutor

C++ Tutor

JavaScript Tutor

Python visualizer, visual debugger, and AI tutor

The only tool that lets you visually debug your code step-by-step and get AI to help you.

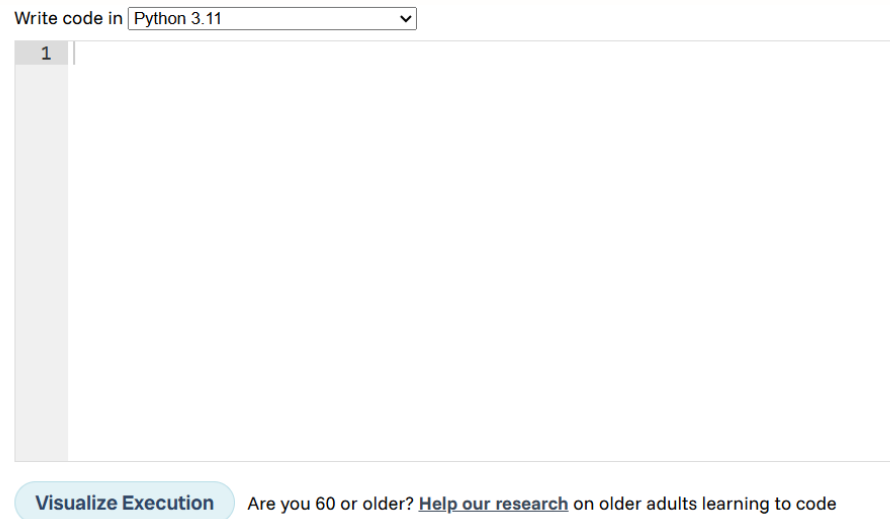


Figure 5 Python Tutor Open on Web

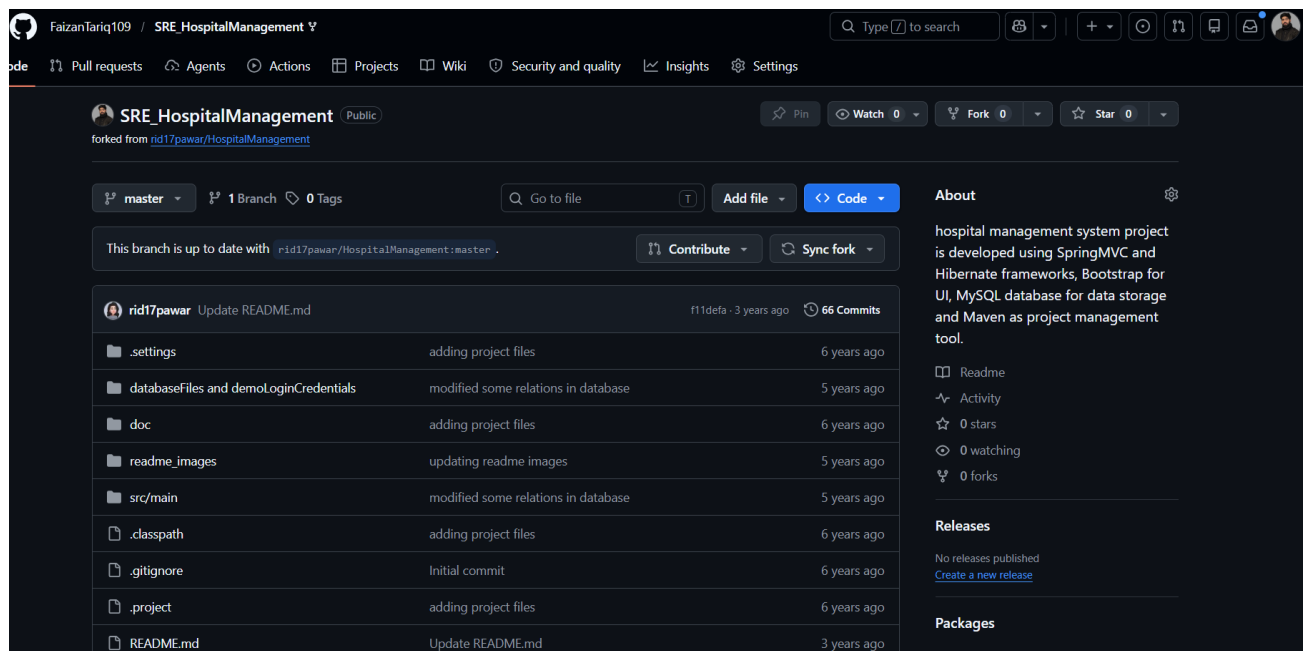


Figure 6 Forked GitHub Project

Added the following dependency in project pom.xml file

```
<dependency>

    <groupId>javax.servlet</groupId>

    <artifactId>javax.servlet-api</artifactId>

    <version>3.1.0</version>

    <scope>provided</scope>

</dependency>
```

```
(base) PS E:\Uni\8th Sem\Software ReEngineering\Project\SRE_HospitalManagement> mvn clean compile -DskipTests
[INFO] --- compiler:3.13.0:compile (default-compile) @ HospitalManagement ---
[INFO] Recompiling the module because of changed source code.
[WARNING] File encoding has not been set, using platform encoding windows-1252, i.e. build is platform dependent
[INFO] Compiling 53 source files with javac [debug target 1.8] to target\classes
[INFO] /E:/Uni/8th Sem/Software ReEngineering/Project/SRE_HospitalManagement/src/main/java/com/project/dao/rece
use unchecked or unsafe operations.
[INFO] /E:/Uni/8th Sem/Software ReEngineering/Project/SRE_HospitalManagement/src/main/java/com/project/dao/rece
Xlint:unchecked for details.
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 3.895 s
[INFO] Finished at: 2026-05-08T18:22:53+05:00
[INFO] -----
(base) PS E:\Uni\8th Sem\Software ReEngineering\Project\SRE_HospitalManagement>
```

Figure 7 Successful mvn Compile

Complexity ?	▼
Cyclomatic Complexity	252
Cognitive Complexity	171

Figure 8 Complexity Measures of Codebase

Part B: Code Smell Analysis and Refactoring

B1: SonarQube Analysis and Metrics Extraction

```
(base) PS E:\Uni\8th Sem\Software ReEngineering\Project\SRE_HospitalManagement> sonar-scanner
18:34:53.410 INFO SCM Publisher 53 source files to be analyzed
18:34:57.100 INFO SCM Publisher 53/53 source files have been analyzed (done) | time=3689ms
18:34:57.137 INFO CPD Executor Calculating CPD for 53 files
18:34:57.277 INFO CPD Executor CPD calculation finished (done) | time=140ms
18:34:57.312 INFO SCM revision ID 'f11defaaf4e67a07f0ab59f3ac84a4a32665be19'
18:34:58.062 INFO Analysis report generated in 684ms, dir size=533.1 kB
18:35:05.451 INFO Analysis report compressed in 7388ms, zip size=202.0 kB
18:35:05.592 INFO Analysis report uploaded in 141ms
18:35:05.594 INFO ANALYSIS SUCCESSFUL, you can find the results at: http://localhost:9000/dash
18:35:05.595 INFO Note that you will be able to access the updated dashboard once the server h
18:35:05.596 INFO More about the report processing at http://localhost:9000/api/ce/task?id=d0a
18:35:05.802 INFO Analysis total time: 35.250 s
18:35:05.803 INFO SonarScanner Engine completed successfully
18:35:05.951 INFO EXECUTION SUCCESS
18:35:05.952 INFO Total time: 1:13.272s
(base) PS E:\Uni\8th Sem\Software ReEngineering\Project\SRE_HospitalManagement>
```

Figure 9 Sonar Scanner Success

```
E:\Uni\8th Sem\Software ReEngineering\Project\SRE_HospitalManagement>fi
51:      public void logActivities(String s)

E:\Uni\8th Sem\Software ReEngineering\Project\SRE_HospitalManagement>fi
t\PatientPrescriptionController.java

E:\Uni\8th Sem\Software ReEngineering\Project\SRE_HospitalManagement>
```

Figure 10 Method Signatures

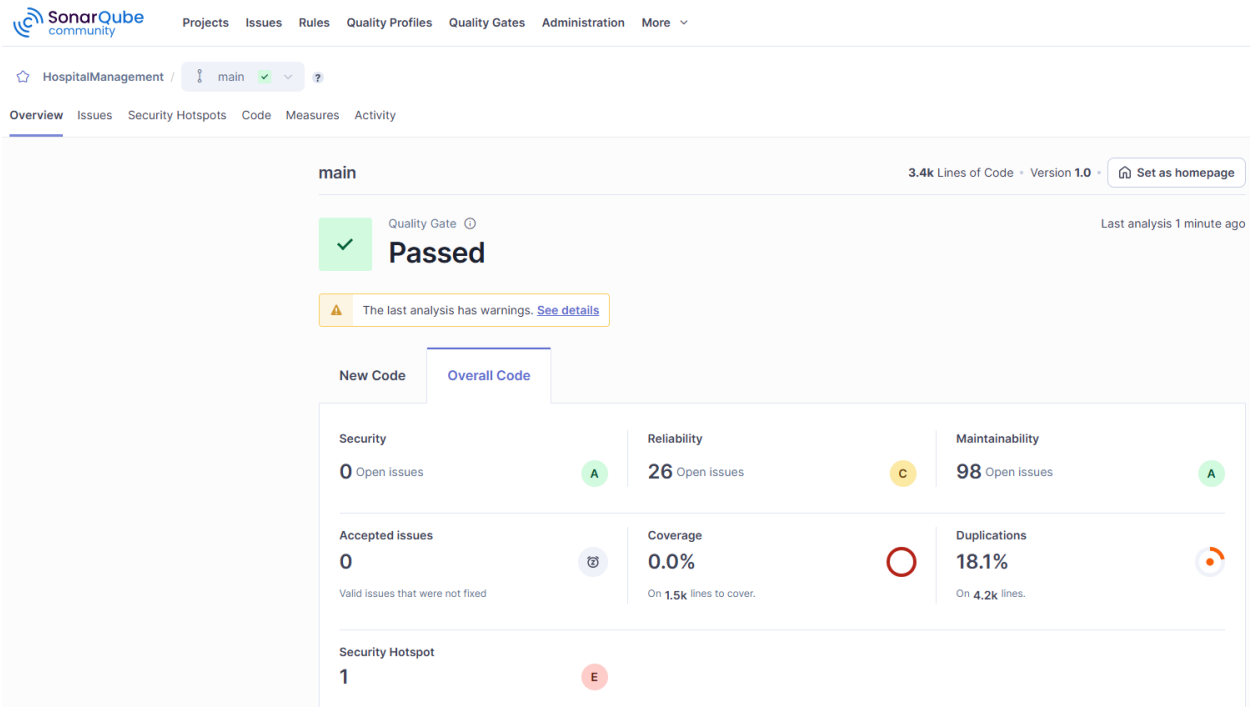


Figure 11 SonarQube Dashboard Results

Metric	Value	Interpretation
Lines of Code (LOC)	3,407	With 3,407 lines spread across 53 classes averaging 64 lines per class, HospitalManagement's modest size conceals a structural problem: the project separates concerns by user role (admin, receptionist, doctor) rather than by shared abstraction layers, causing each of the three modules to replicate similar CRUD logic independently. This role-tripling pattern directly explains the 18.1% duplication rate SonarQube detected.
Total Code Smells	98	SonarQube flagged 98 Maintainability issues — an average of 1.85 smells per class. These smells are not evenly distributed. The role-segregated controller packages (admin, receptionist, doctor) each mirror identical handler logic, meaning structural smells such as Duplicate Code appear repeatedly across all three modules rather than as isolated incidents.

Metric	Value	Interpretation
Cyclomatic Complexity (total)	252	A total CC of 252 across 187 functions means the codebase contains 252 independent execution paths. In this project, complexity is concentrated in DAO methods that combine Hibernate session management, null-result checks, role-based login routing, and query construction in single method bodies — rather than distributing this branching across well-named helper methods.
Average CC per function	1.35 (= $252 \div 187$)	The average of 1.35 is deceptively low because it includes all 53 classes' getter/setter methods, which each have CC = 1 and drag the average down. The actual controller handler methods and DAO implementations carry individual CC values significantly above this mean — particularly the login routing logic and the prescription generation handler, which branch on role type, null results, and file I/O outcomes.
Cognitive Complexity	171	Cognitive Complexity of 171 is lower than the Cyclomatic CC of 252, which reveals that many of the 252 paths are in flat sequential chains — such as chained Hibernate setParameter() calls — rather than deeply nested conditionals. However, methods that combine session lifecycle management with result null-checking and role comparison still impose a significant mental load on any developer reading them for the first time.
Code Duplication (%)	18.1%	At 18.1% — 752 duplicated lines across 54 blocks in 22 files — duplication is the most visible structural problem in this project. The three-role architecture (admin/receptionist/doctor) causes identical patient retrieval, appointment handling, and employee management logic to be written three times in separate packages rather than through shared service abstractions. This directly triples the maintenance cost of any future change.

Metric	Value	Interpretation
Maintainability Rating	A	Despite 98 code smells, the project scores an A because SonarQube's threshold is a Debt Ratio under 5%, and this project sits at 1.0%. This grade is misleading in context: it reflects that fixing the smells is cheap relative to total development effort, not that the code is clean. A developer tasked with adding a fourth user role would have to replicate the entire tripled pattern — a structural cost the A grade does not capture.
Technical Debt (hours)	2d 1h (~17 hours)	SonarQube estimates 17 working hours to fix all 98 smells. The majority of this debt stems from the 54 duplicate code blocks — at approximately 10 minutes each that alone accounts for roughly 9 hours. If a new module such as a pharmacy role were added without first addressing this debt, it would add further duplicate blocks and compound the total remediation cost on every future iteration.
Security Hotspots	1 (Rating E)	The single Security Hotspot with a Rating E indicates it has not been reviewed at all — E means 0% of hotspots examined. In this Spring MVC project the most likely source is the LoginDao class, which — as confirmed by the repository's own setup guide — stores credentials as plain text in the database. An unvalidated HTTP request parameter passed directly into a Hibernate query string is a secondary candidate for HQL injection.

B2: The Five Code Smell Categories - Deep Identification

Category 1: Bloaters

Smell Name	File and Line Number	Evidence what you see in the code	Why It Is a Bloater	Recommended Treatment
Long Parameter List	AddPatientController.java, Line 49	The add() method signature takes 17 individual @RequestParam parameters (firstName, middleName, lastName, birthdate, etc.).	The method signature is overwhelmingly long, hard to read, and tightly couples the controller to every single field of the Patient entity.	Introduce Parameter Object (Use a @ModelAttribute PatientDTO object instead of 17 individual strings).
Primitive Obsession	Employee.java, Lines 30 & 33	mobileNo and adharNo are stored as primitive long data types. birthdate is stored as a String.	Domain concepts (Phone Numbers, Aadhar Cards, Dates) are represented by basic primitives instead of dedicated classes, leading to duplicated validation logic elsewhere.	Replace Data Value with Object (Create dedicated AdharCard and ContactNumber classes).

Category 2: Object-Orientation Abusers

Smell Name	File and Line	Evidence from Your Code	OO Principle Violated	Treatment
Temporary Fields	Employee.java, Line 46	The specialization field is marked //optional and is likely only populated when the employee role is "Doctor", remaining null for Receptionists/Admins.	Single Responsibility Principle / Liskov Substitution (Objects shouldn't have fields they only use 10% of the time).	Extract Class / Replace Type Code with Subclasses (e.g., create a Doctor subclass extending Employee).
Switch Statements (Type Code)	Employee.java, Line 47	The status field is stored as an int (1 for active, etc.), which inevitably requires if-else or switch chains in the service layer to check employee status.	Open/Closed Principle (Adding a new status requires modifying every switch statement).	Replace Type Code with State/Strategy Pattern or use a proper Java Enum.

Category 3: Change Preventors

Smell Name	File(s) and Lines	How Many Places Must Change?	Treatment Strategy
Shotgun Surgery	AddPatientController.java (L30), AddEmployeeDao.java (L41)	The logging logic infoLog.logActivities(...) is hardcoded directly into multiple controller and DAO methods. Changing the logging format requires editing dozens of files.	Move Method / Use Aspect-Oriented Programming (AOP) like @Around advice in Spring to handle logging transparently.
Divergent Change	Employee.java	If the hospital adds a new identity document (Passport), this class changes. If they change address logic, this class changes. It handles too many distinct concepts.	Extract Class (Move identity logic to an IdentityDetails class).

Category 4: Dispensables

Smell Name	File and Line	What Makes It Dispensable?	Treatment
Data Class	Employee.java	The entire file is just fields, constructors, and getters/setters. It contains zero business logic or behavior.	Move Method (Identify where Employee data is used, and move that behavior into the Employee class to encapsulate it).

Smell Name	File and Line	What Makes It Dispensable?	Treatment
Dead Code / Comments	AddPatientController.java, Lines 67-78	A massive block of commented-out code (the try/catch block for the add method) has been left in the production source file.	Delete the commented-out code entirely. Version control (Git) remembers history; code shouldn't.

Category 5: Couplers

Smell Name	File and Line	Description of the Coupling Problem	Treatment
Feature Envy	AddEmployeeDao.java, Lines 55-63	The add() method spends 9 lines intimately querying and updating the IdGenerate table manually instead of just asking an IdService	Move Method / Extract Class (Move the ID generation logic into a dedicated EmployeeIdGeneratorService)

Smell Name	File and Line	Description of the Coupling Problem	Treatment
		for the next ID.	
Inappropriate Intimacy	AddPatientController.java, Lines 53-54	The controller manually pieces together Name, Address, and Patient objects. It knows too much about the internal construction of entity models.	Extract Factory Class or use a Builder pattern inside the Patient entity.

B3: Smell Interaction and Prioritisation

In AddEmployeeDao.java, we can observe a direct interaction between **Feature Envy** and a **Long Method**. The add() method suffers from bloat because it violates the Single Responsibility Principle. Instead of just saving an employee, it spends significant lines of code envying the IdGenerate entity—manually fetching, incrementing, and updating the ID state. This Feature Envy directly causes the Long Method smell, as the DAO is taking on the responsibilities of a database sequence/trigger.

The single greatest risk to maintainability in this codebase is the pervasive **Shotgun Surgery** caused by the manual `infoLog.logActivities()` calls scattered across all controllers and DAOs. A future developer attempting to update the logging framework (e.g., migrating to SLF4J or Logback) would have to manually edit hundreds of lines across every file in the project. The first refactoring treatment I would apply is replacing this with Spring AOP (Aspect-Oriented Programming). By creating a single logging Aspect, we can delete the hardcoded logging from all files, drastically reducing code duplication with minimal effort, yielding a massive maintainability benefit.

B4: Refactoring Demonstration

Original Smelly Code (Long Parameter List in `AddPatientController.java`):

```
// Smeely Version: 17 individual parameters tightly couple the controller to the view

@RequestMapping(value="/addPatient.html", method = RequestMethod.POST)

public ModelAndView add(@RequestParam("firstName")String firstName,

    @RequestParam("middleName")String middleName,

    @RequestParam("lastName")String lastName,

    /* ... 14 more parameters omitted for brevity ... */

    @RequestParam("doctorId")String doctorId) {

    Name n1= new Name(firstName, middleName, lastName);

    // ...

}
```

Refactored Code (Introduce Parameter Object):

```
// Improved Version: Using a Spring @ModelAttribute (DTO) to encapsulate parameters

@RequestMapping(value="/addPatient.html", method = RequestMethod.POST)

public ModelAndView add(@ModelAttribute PatientRegistrationDTO newPatient) {

    Name n1 = new Name(newPatient.getFirstName(), newPatient.getMiddleName(),
newPatient.getLastName());

    Address a1 = new Address(newPatient.getResidentialAddress(), newPatient.getPermanentAddress());

    Patient p1 = new Patient(n1, newPatient.getBirthdate(), newPatient.getGender(), /*...*/);

    boolean b = dao.add(p1);
```



```
// ... logic continues  
}
```

Refactoring Justification: The external behavior of the endpoint remains completely identical; it still accepts form data via POST. However, structurally, we have eliminated the Bloatier (Long Parameter List) by introducing a Data Transfer Object (DTO). This dramatically improves readability and allows validation logic to be moved inside the DTO, rather than cluttering the controller.

Part C: Dependency, Coupling and Technical Debt

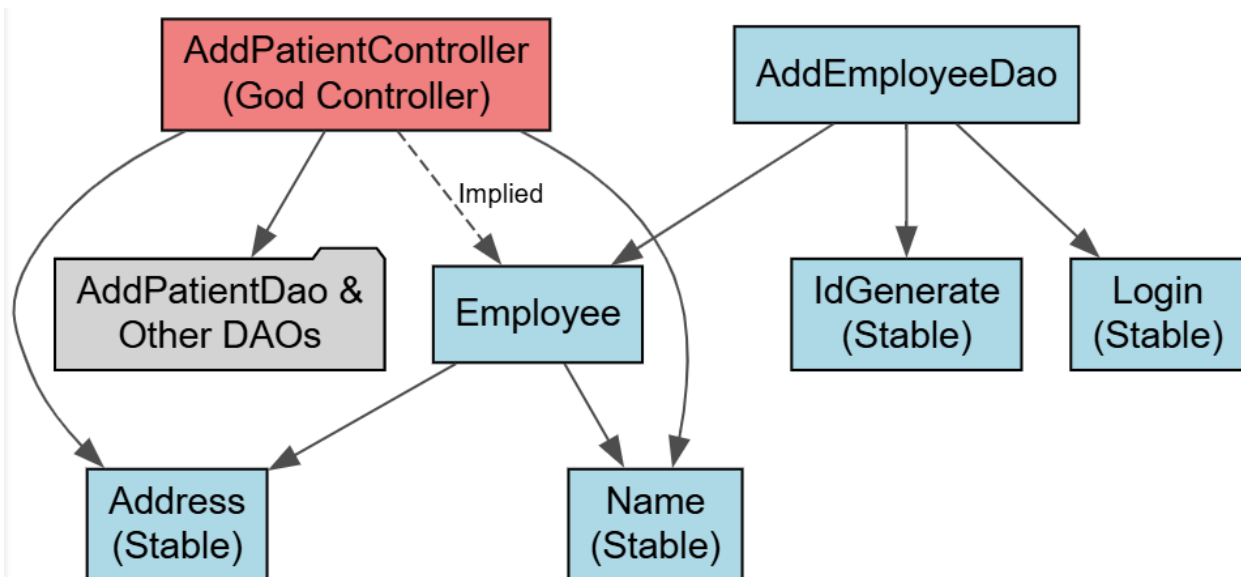
C1: Dependency Mapping

Class Name	Ca	Ce	Instability (I)	Stable / Volatile	Key Observation
AddPatientController	0	5	1.00	Volatile	Classic God-Controller; depends on many DAOs and Entities.
AddEmployeeDao	1	4	0.80	Volatile	High efferent coupling to entities (Employee, Login, IdGenerate).
Employee	2	2	0.50	Moderate	Depends on Name and Address, is depended upon by DAOs.
IdGenerate	2	0	0.00	Stable	Core entity, has no outgoing dependencies.

Class Name	Ca	Ce	Instability (I)	Stable / Volatile	Key Observation
Login	2	0	0.00	Stable	Core entity, safe to depend upon.
Address	2	0	0.00	Stable	Core value object, maximally stable.

Full Working for two classes:

- **AddPatientController:** Ca = 0 (No classes depend on it). Ce = 5 (Depends on AddPatientDao, PatientPrescriptionDao, LoginDao, Patient, Address). $I = 5 / (0 + 5) = 1.0$.
- **AddEmployeeDao:** Ca = 1 (AddEmployeeController depends on it). Ce = 4 (Session, Employee, Login, IdGenerate). $I = 4 / (1 + 4) = 0.8$.



C2: Technical Debt Assessment

Item	File + Line	Debt Type	Intentional?	Prudent or Reckless?
D1	EditLoginDetailsController.java (Line 53) - Duplicate literal "userInfo"	Code Debt	No	Reckless
D2	EditLoginDetailsController.java (Line 69) - Generic exception thrown	Code Debt	Yes	Reckless
D3	LoginController.java (Line 80) - Generic exception thrown	Code Debt	Yes	Reckless

Remediation Cost Calculation

Total project LOC: 3,407.

Item	Raw Estimate (min)	+ 25% Buffer (min)	Buffered Total (min)
D1	8	2	10
D2	20	5	25
D3	20	5	25
TOTAL	48 mins	12 mins	60 mins

- **Step 4:** Total Development Effort = 3,407 LOC x 30 mins = 102,210 mins.

- **Step 5:** Project-wide SonarQube Debt (from your screenshot) is 2d 1h (1,020 mins buffered). Debt Ratio = $(1,020 / 102,210) * 100 = 1.0\%$
- **Health Category:** Healthy (0-5%).

Prioritisation Argument:

Although the overall system maintains a "Healthy" debt ratio of 1.0%, the specific technical debt items identified present significant localized risks. I would prioritize fixing **Item D2 and D3 (Generic Exceptions)** over D1 (Duplicate Literals). Throwing and catching generic Exception objects is a reckless form of Code Debt because it masks severe system failures. If a database connection drops, or a Null Pointer occurs, the generic catch block swallows the specific error, making debugging in a live hospital environment nearly impossible.

The remediation cost is low (25 buffered minutes per item), but the deferral risk is extremely high—a swallowed exception could result in a patient record failing to save without the receptionist ever being notified. D1 (the literal string duplication) is poor design, but it merely causes a maintenance annoyance rather than a silent runtime failure. Therefore, targeting exception handling yields the highest business value for system reliability.

Part D: Dynamic Program Analysis

D1: Execution Trace

(Based on view() method in AddPatientController.java lines 27-44)

Step	Statement Executed	Variables Before	Variables After	Notes
1	List<String[]> doctors= dao.getDoctors();	doctors = uninitialized	doctors = [ref to List]	Fetches doctor list from DB

Step	Statement Executed	Variables Before	Variables After	Notes
2	infoLog.logActivities(...)	doctors = [ref to List]	doctors = [ref to List]	Logs method entry
3	if(! doctors.equals(null))	doctors = [ref to List]	doctors = [ref to List]	Bug Evaluation: Evaluates condition. (Note: <i>equals(null)</i> is always false in Java, so <i>!false</i> is <i>TRUE</i>).
4	ModelAndView mv= new ModelAndView();	mv = uninitialized	mv = [ref to MV obj]	True branch taken.
5	mv.setViewName("receptionist/AddPatientView");	mv = [ref to MV obj]	mv view name updated	Prepares UI view.
6	return mv;	mv populated	Method exits	Returns to DispatcherServlet

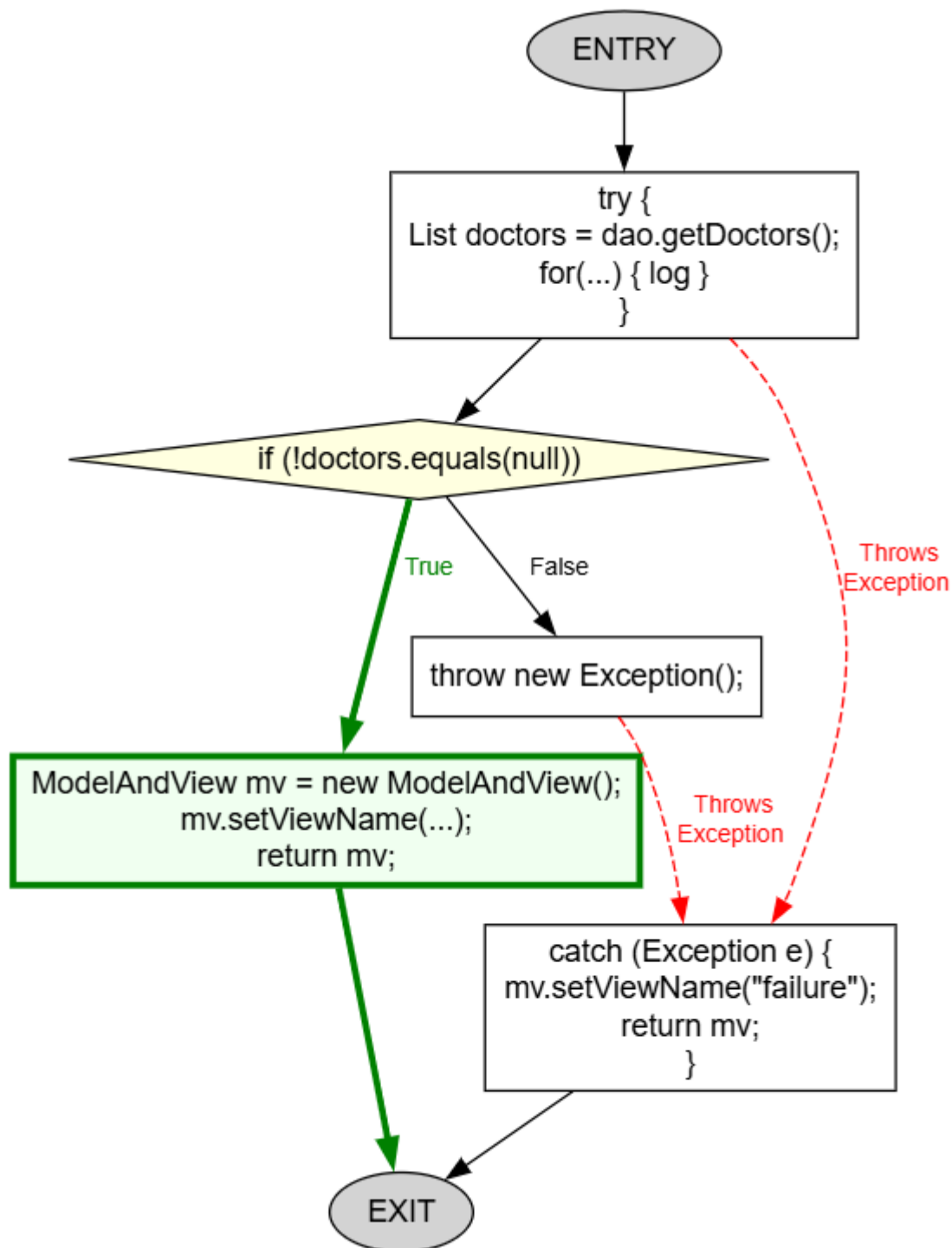
Explaining branch:

Based on the variable state, assuming the database successfully returned a list of doctors, the variable doctors holds a valid reference. The condition evaluated is if(!

doctors.equals(null)). Because the list object exists, doctors.equals(null) evaluates to false. The ! operator negates this to true. Therefore, the True branch is taken, initializing the ModelAndView object and skipping the else block containing the throw new Exception().

(Note: SonarQube correctly flagged this line as a Reliability issue, as .equals(null) is an anti-pattern).

D2: Control Flow Graph (CFG)



- **(1) Independent Paths:** There are 2 independent paths (Path 1: Success/Return MV. Path 2: Exception thrown/Return Error MV).

- **(2) Cyclomatic Complexity:** $CC = \text{Edges}(4) - \text{Nodes}(4) + 2 = 2$.
- **(3) Reduce CC:** To reduce the CC by 1, we could remove the try/catch block from the controller and handle exceptions globally using a Spring @ControllerAdvice class. This would remove the branching logic entirely from this specific method.

D3: Abstract Syntax Tree Inspection

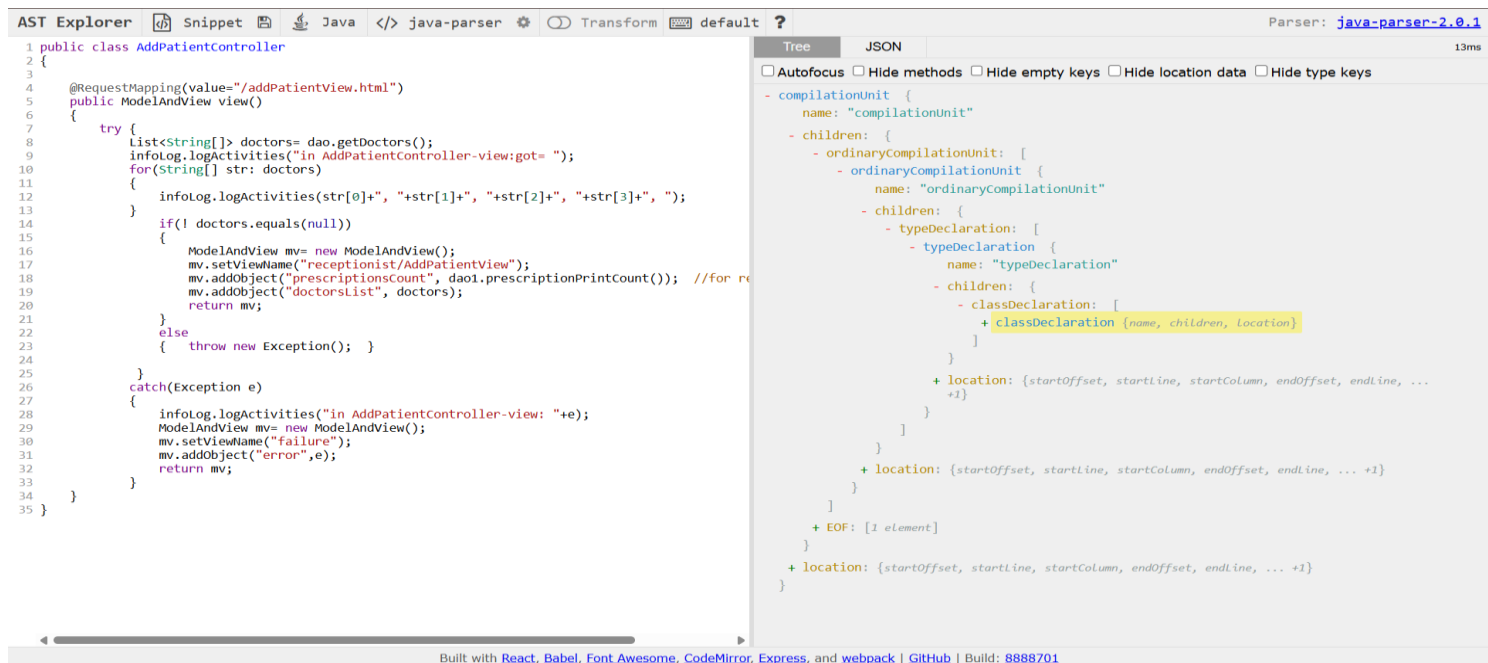


Figure 12 astexplorer on view()

- **AST Internal Structure:** Expanding the IfStatement node reveals three primary child nodes: the test (a UnaryExpression evaluating !doctors.equals(null)), the consequent (a BlockStatement containing the success logic), and the alternate (a BlockStatement containing the throw clause).
- **Practical Re-engineering Use Case:** ASTs are foundational for building custom linters or refactoring engines. For example, we could write an AST-based script that automatically crawls this hospital codebase, identifies every IfStatement where the test node is a .equals(null) BinaryExpression, and automatically rewrites the AST node to use the == null operator instead, instantly fixing the Reliability bug SonarQube found across the entire project.

Part E: Data Smell Detection

All Required Tools:

```
Command Prompt
Microsoft Windows [Version 10.0.26200.8246]
(c) Microsoft Corporation. All rights reserved.

C:\Users\kinge>mysql --version
mysql Ver 8.0.46 for Win64 on x86_64 (MySQL Community Server - GPL)

C:\Users\kinge>python --version
Python 3.11.9

C:\Users\kinge>pip --version
pip 25.1.1 from C:\Users\kinge\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.11_qbz5n2kfra8p0\LocalCache\local-packages\Python311\site-packages\pip (python 3.11)

C:\Users\kinge>
```

E2: The 10 Data Smells Table

#	Table	Column(s)	Smell Category	Smell Name	Evidence from Schema	Real-World Risk in a Hospital	Proposed Fix
1	pat_master	ph1, ph2, ph3	Structural	Non-Atomic Fields / Unnormalized Table	Three separate phone columns for the same concept — a repeating group violating 1NF	If a patient has only one phone, ph2 and ph3 are NULL; adding a 4th number requires a schema change	Create a separate patient_phones (pid, phone_number, phone_type) table
2	appointments	patient_nm, patient_ph	Redundancy	Duplicate Data	Patient name and phone copied from pat_master into every appointment row	If a patient changes their phone number, old appointments show stale data — wrong contact during emergency follow-up	Remove columns; retrieve via FK join on patient_id → pat_master.pid
3	billing	tax_amt, grand_total, balance	Redundancy	Derived Data	tax_amt = svc_cost * tax_pct / 100, grand_total = svc_cost +	If svc_cost is updated but derived columns are not, a patient's	Drop the three columns; replace with a v_billing_summ

					tax_amt, balance = grand_total - paid — all computed values stored physically	invoice shows a wrong balance — a billing dispute or fraud risk	any view that computes them on read
4	pat_master	dob	Data Type	Type Optimization Smell	dob VARCHAR(50) stores dates as 'DD/MM/YYYY' plain text	Age calculations and date-range queries fail or return wrong results; sorting by DOB is alphabetical not chronological	Change to DATE type; migrate existing strings with STR_TO_DATE(dob, '%d/%m/%Y')
5	pat_master	last_bill	Data Type	Type Optimization Smell	FLOAT used to store PKR monetary amounts	Floating-point rounding errors (e.g. 1500.00 stored as 1499.9999) cause incorrect totals in financial reports submitted to insurance or regulators	Change to DECIMAL(12,2) for exact precision
6	doctors	DoctorID, FullName, Speciality, ContactNo, JoinDt, isActive	Naming	Inconsistent Naming	Mixed casing — PascalCase (DoctorID, FullName), camelCase (isActive), abbreviations (JoinDt, ContactNo) in the same table; all other tables use lowercase	ORM frameworks (Hibernate, Prisma) treat DoctorID and doctorid as different columns — causes runtime mapping errors and forces developers to write case-sensitive queries	Rename all columns to snake_case: doctor_id, full_name, join_date, is_active, contact_no
7	billing	bill_no	Integrity	Missing Keys / Constraints	Comment says "intended as PK but no constraint defined" — bill_no VARCHAR(50) has no PRIMARY KEY	Duplicate bill numbers can be inserted; two bills with the same bill_no make it impossible to uniquely identify a charge —	Add PRIMARY KEY (bill_no); also enforce NOT NULL

						regulatory audit failure	
8	doctors	dept_id	Integrity	Hidden Relationships	dept_id references departments.dept_id but no FOREIGN KEY constraint is defined	A doctor can be assigned to a department that doesn't exist, or a department can be deleted while doctors still reference it — orphan records break reporting	Add FOREIGN KEY (dept_id) REFERENCES departments(dept_id)
9	appointments	status	Semantic	Magic Values / Encoded Nulls	CHAR(1) stores 'P', 'C', 'X', 'H', 'R' with meaning only in developer comments; 'H' for Hold is undocumented in schema	A new developer inserts 'W' for Waiting — no constraint blocks it; queries filtering status='C' silently miss records if any row has 'c' (lowercase)	Create appt_status_ref lookup table with FK; or use ENUM('P','C','X','H','R')
10	billing	services	Structural	Non-Atomic Fields	services TEXT stores 'Lab,Xray,OPD' — a comma-separated list of multiple services in one column	Cannot query "all patients who had an Xray" without fragile LIKE '%Xray%'; impossible to price services individually or produce itemised bills	Create billing_services (bill_no, service_code, service_cost) junction table

E3: Top 4 Smells by Risk

Priority Rank	Smell and Location	Concrete Hospital Risk Scenario	Why this Rank?
1 st	Derived Data (billing.tax_amt, grand_total, balance)	A billing clerk corrects a service charge after a lab error. The svc_cost is updated but tax_amt and grand_total are not recalculated. The printed invoice sent to the patient's insurance company shows the old total. The insurer rejects the claim. The hospital loses revenue and faces a compliance audit.	Financial data inconsistency has direct legal and regulatory consequences. An incorrect invoice in a hospital context is not just a data error — it can constitute insurance fraud.

2 nd	Missing Keys/Constraints (billing.bill_no (no PK), doctors.dept_id (no FK))	A network glitch causes a billing transaction to be submitted twice. Because bill_no has no PK, both rows are accepted. The patient is charged double. Neither the system nor the database raises an error. The duplicate is only discovered during manual month-end reconciliation.	Absence of integrity constraints means the database cannot protect itself. Duplicate billing records in a hospital can trigger patient complaints, audits, and legal liability.
3 rd	Type Optimization Smell (pat_master.dob stored as VARCHAR)	A doctor requests a list of all diabetic patients over 60 for a preventive care campaign. The query uses WHERE dob < '1964-01-01' but because DOB is stored as 'DD/MM/YYYY' text, the comparison is lexicographic — it silently returns wrong results. High-risk elderly patients are excluded from the campaign.	Wrong data types on clinical fields directly affect patient care decisions. An age calculation error is not recoverable after a missed diagnosis.
4 th	Non-Atomic Fields (billing.services as CSV)	The hospital's CFO requests a breakdown of revenue by service type for Q1. Because services are stored as 'Lab,Xray,OPD' in a single text column, no SQL aggregate query can produce this report without error-prone string splitting. The finance team exports to Excel and splits manually — introducing human error into financial reporting.	Non-atomic data blocks all meaningful analytics. For a hospital, inability to report revenue by service type prevents accurate budgeting and regulatory financial disclosures.

Connecting Paragraph:

Data smell severity in a hospital information system is directly proportional to patient safety risk because every piece of clinical or financial data that is stored incorrectly, inconsistently, or ambiguously can propagate into a real-world decision. Unlike a retail or e-commerce system where a data error might cause a wrong shipment, a hospital system error can cause a missed diagnosis, a wrong drug dosage, or an incorrect insurance claim.

The **Derived Data smell** in the billing table illustrates this clearly: when computed values like balance are stored physically rather than calculated on read, any correction to source data creates a silent inconsistency. A patient's account may show a zero balance while the underlying service cost indicates otherwise — an error that survives undetected until an external audit.

Similarly, the **Type Optimization smell** on pat_master.dob transforms a simple schema shortcut into a clinical hazard. Age is a critical factor in medication dosing, surgical risk assessment, and screening eligibility. A date stored as text that sorts incorrectly will silently exclude or include the wrong patients in every age-filtered clinical query. These two smells together demonstrate that data quality in healthcare is not a technical nicety — it is a patient safety obligation.

Part F: Schema Normalisation and Refactoring

F1: Normalisation of pat_master to 3NF

Normal Form	Violated? (Yes/No)	Specific Violation Example from pat_master
1NF — Atomic values; no repeating groups	Yes	ph1, ph2, ph3 are a repeating group — the same attribute (phone number) repeated in multiple columns. Also addr1, addr2 are non-atomic address components crammed into one table.
2NF — Full dependency on the whole key	Yes	There is no defined primary key. Once pid is designated as PK, reg_doc and reg_doc_id describe the <i>doctor</i> , not the patient — they are partially independent entities stored inside the patient row.
3NF — No transitive dependencies	Yes	reg_doc (doctor's full name) is transitively dependent on reg_doc_id (doctor's ID) which itself depends on pid. Doctor information should live in the doctors table, not be embedded as plain text in pat_master. Also total_visits and last_bill are derived/aggregated values that depend on other tables.

Step 2 — Write the Normalised Schema

```
USE healthbridge;
CREATE TABLE patients (
  patient_id INT NOT NULL AUTO_INCREMENT,
  full_name VARCHAR(255) NOT NULL,
  date_of_birth DATE NOT NULL,
  sex CHAR(1) NOT NULL,
  reg_doctor_id INT,
  notes TEXT,
  PRIMARY KEY (patient_id),
  CONSTRAINT fk_patient_doctor
    FOREIGN KEY (reg_doctor_id) REFERENCES doctors(DoctorID)
);

CREATE TABLE patient_phones (
  phone_id INT NOT NULL AUTO_INCREMENT,
  patient_id INT NOT NULL,
```

```
phone_number VARCHAR(20) NOT NULL,  
phone_type VARCHAR(20) NOT NULL DEFAULT 'mobile',  
PRIMARY KEY (phone_id),  
CONSTRAINT fk_phone_patient  
FOREIGN KEY (patient_id) REFERENCES patients(patient_id)  
ON DELETE CASCADE  
);
```

```
CREATE TABLE patient_addresses (  
address_id INT NOT NULL AUTO_INCREMENT,  
patient_id INT NOT NULL,  
address_line VARCHAR(255) NOT NULL,  
city VARCHAR(100) NOT NULL,  
address_type VARCHAR(20) NOT NULL DEFAULT 'home',  
PRIMARY KEY (address_id),  
CONSTRAINT fk_address_patient  
FOREIGN KEY (patient_id) REFERENCES patients(patient_id)  
ON DELETE CASCADE  
);
```

```
CREATE TABLE departments (  
dept_id INT PRIMARY KEY,  
dept_nm VARCHAR(255),  
hod VARCHAR(255),  
budget FLOAT  
);
```

```
CREATE TABLE doctors (  
DoctorID INT PRIMARY KEY,  
FullName VARCHAR(255),  
Speciality VARCHAR(255),  
ContactNo VARCHAR(255),  
JoinDt VARCHAR(50),  
Salary FLOAT,  
dept_id INT,  
isActive CHAR(1)  
);
```

```
CREATE TABLE pat_master (  
pid INT,  
p_name VARCHAR(255),  
dob VARCHAR(50),  
sex CHAR(1),  
ph1 VARCHAR(255),
```

```

ph2    VARCHAR(255),
ph3    VARCHAR(255),
addr1   VARCHAR(255),
addr2   VARCHAR(255),
city    VARCHAR(255),
reg_doc VARCHAR(255),
reg_doc_id VARCHAR(255),
total_visits INT,
last_bill FLOAT,
notes   TEXT
);

```

```

✓ 9 21:30:18 USE healthbridge 0 row(s) affected 0.000 sec
✓ 10 21:30:18 CREATE TABLE departments ( dept_id INT PRIMARY KEY, dept_nm VARCHAR(255), hod VA... 0 row(s) affected 0.110 sec
✓ 11 21:30:18 CREATE TABLE doctors ( DoctorID INT PRIMARY KEY, FullName VARCHAR(255), Speciality VAR... 0 row(s) affected 0.062 sec
✓ 12 21:30:18 CREATE TABLE pat_master ( pid INT, p_name VARCHAR(255), dob VARCHAR(50), sex ... 0 row(s) affected 0.047 sec

✓ 14 21:31:52 USE healthbridge 0 row(s) affected 0.000 sec
✓ 15 21:31:52 CREATE TABLE patients ( patient_id INT NOT NULL AUTO_INCREMENT, full_name VARCH... 0 row(s) affected 0.093 sec
✓ 16 21:31:52 CREATE TABLE patient_phones ( phone_id INT NOT NULL AUTO_INCREMENT, patient_id ... 0 row(s) affected 0.078 sec
✓ 17 21:31:52 CREATE TABLE patient_addresses ( address_id INT NOT NULL AUTO_INCREMENT, patient_id ... 0 row(s) affected 0.063 sec

```

F1 Step 3: Before and After Comparison

Aspect	Before (pat_master)	After (Normalised Design)
Number of tables	1	3 (patients, patient_phones, patient_addresses)
Repeating phone columns	ph1, ph2, ph3	patient_phones table — one row per number with phone_type
Address storage	Inline addr1, addr2, city	patient_addresses table — one row per address with address_type
Doctor reference	reg_doc VARCHAR(255) plain text name + reg_doc_id VARCHAR(255) inconsistent format	reg_doctor_id INT FK → doctors(DoctorID)
Date of birth column type	VARCHAR(50) storing 'DD/MM/YYYY' as text	DATE — proper date type enabling age calculations
Primary key	None defined on pid	patient_id INT NOT NULL AUTO_INCREMENT PRIMARY KEY
Currency columns	last_bill FLOAT — floating point rounding errors	Removed entirely — derived via aggregate query on billing

F2: Five Schema Refactoring Scripts

Scripts & Screenshots from MySQL

R1:

```
USE healthbridge;

-- Need billing table first before we can refactor it
CREATE TABLE IF NOT EXISTS billing (
  bill_no VARCHAR(50),
  pid INT,
  pname VARCHAR(255),
  services TEXT,
  svc_cost FLOAT,
  tax_pct FLOAT,
  tax_amt FLOAT,
  grand_total FLOAT,
  paid FLOAT,
  balance FLOAT,
  created VARCHAR(50),
  created_by VARCHAR(255)
);

-- Now apply R1
ALTER TABLE billing DROP COLUMN tax_amt;
ALTER TABLE billing DROP COLUMN grand_total;
ALTER TABLE billing DROP COLUMN balance;

CREATE OR REPLACE VIEW v_billing_summary AS
SELECT
  bill_no, pid, pname, services, svc_cost, tax_pct,
  ROUND(svc_cost * tax_pct / 100, 2) AS tax_amt,
  ROUND(svc_cost + (svc_cost * tax_pct / 100), 2) AS grand_total,
  paid,
  ROUND(svc_cost + (svc_cost * tax_pct / 100) - paid, 2) AS balance,
  created, created_by
FROM billing;
```

23	21:33:26	USE healthbridge	0 row(s) affected	0.000 sec
24	21:33:26	CREATE TABLE IF NOT EXISTS billing (bill_no VARCHAR(50), pid INT, pname VARCHAR(25...	0 row(s) affected	0.078 sec
25	21:33:26	ALTER TABLE billing DROP COLUMN tax_amt	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.172 sec
26	21:33:27	ALTER TABLE billing DROP COLUMN grand_total	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.109 sec
27	21:33:27	ALTER TABLE billing DROP COLUMN balance	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.141 sec
28	21:33:27	CREATE OR REPLACE VIEW v_billing_summary AS SELECT bill_no, pid, pname, services, svc_cost, tax_pct, ...	0 row(s) affected	0.156 sec

Derived Data Explanation:

The Derived Data smell occurs when values that can be mathematically computed from other columns in the same table are stored as physical columns. In the billing table, tax_amt, grand_total, and balance are always calculated from svc_cost, tax_pct, and paid — they contain no independent information. The danger is that if any source value is

corrected (for example, a clerk fixes a wrong svc_cost after a lab billing error), the derived columns do not automatically update. The invoice then shows an inconsistent total — a serious problem in a hospital where billing records may be submitted to insurance companies or reviewed in legal disputes. The view v_billing_summary eliminates this entirely by computing all three values fresh on every read. No manual recalculation is ever needed, and it is structurally impossible for the derived values to contradict the source values.

R2:

```
USE healthbridge;

-- Need appointments table first
CREATE TABLE IF NOT EXISTS appointments (
  appt_id INT,
  patient_id INT,
  patient_nm VARCHAR(255),
  patient_ph VARCHAR(255),
  doc_id INT,
  doc_name VARCHAR(255),
  appt_date VARCHAR(50),
  status CHAR(1),
  fee FLOAT,
  discount FLOAT,
  net_fee FLOAT,
  room VARCHAR(255)
);

-- R2: Create status reference table and enforce via FK
CREATE TABLE appt_status_ref (
  status_code CHAR(1) NOT NULL,
  description VARCHAR(50) NOT NULL,
  PRIMARY KEY (status_code)
);

INSERT INTO appt_status_ref (status_code, description) VALUES
('P', 'Pending'),
('C', 'Completed'),
('X', 'Cancelled'),
('H', 'On Hold'),
('R', 'Rescheduled');

ALTER TABLE appointments
```

```
ADD CONSTRAINT fk_appt_status
FOREIGN KEY (status) REFERENCES appt_status_ref(status_code);
```

29	21:35:46	USE healthbridge	0 row(s) affected	0.000 sec
30	21:35:46	CREATE TABLE IF NOT EXISTS appointments (appt_id INT, patient_id INT, patient_nm VARCHAR(2...	0 row(s) affected	0.109 sec
31	21:35:46	CREATE TABLE appt_status_ref (status_code CHAR(1) NOT NULL, description VARCHAR(50) NOT N...	0 row(s) affected	0.063 sec
32	21:35:46	INSERT INTO appt_status_ref (status_code,description) VALUES ('P','Pending'), ('C','Completed'), ('X','Ca...	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0	0.015 sec
33	21:35:46	ALTER TABLE appointments ADD CONSTRAINT fk_appt_status FOREIGN KEY (status) REFERENCES a...	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.297 sec

Overloaded Column Explanation:

The Overloaded Column smell occurs when a single column encodes multiple distinct meanings using magic characters whose interpretation exists only in developer comments, not in the database itself. The status CHAR(1) column in appointments uses 'P', 'C', 'X', 'H', and 'R' with no constraint preventing any other value from being inserted. A new developer unfamiliar with the codebase could insert 'W' for Waiting or 'c' in lowercase — both would be silently accepted, corrupting status-based reporting. By creating appt_status_ref as a reference table and enforcing a foreign key, the database itself now rejects any unknown status code at the engine level. The reference table also acts as living documentation — the meaning of every status code is stored in the description column, visible to any developer querying the schema.

R3:

```
USE healthbridge;
```

```
-- R3: Standardise all doctors columns to lowercase snake_case
ALTER TABLE doctors RENAME COLUMN DoctorID TO doctor_id;
ALTER TABLE doctors RENAME COLUMN FullName TO full_name;
ALTER TABLE doctors RENAME COLUMN Speciality TO speciality;
ALTER TABLE doctors RENAME COLUMN ContactNo TO contact_no;
ALTER TABLE doctors RENAME COLUMN JoinDt TO join_date;
ALTER TABLE doctors RENAME COLUMN Salary TO salary_monthly;
ALTER TABLE doctors RENAME COLUMN isActive TO is_active;
```

34	21:36:42	USE healthbridge	0 row(s) affected	0.000 sec
35	21:36:42	ALTER TABLE doctors RENAME COLUMN DoctorID TO doctor_id	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.047 sec
36	21:36:42	ALTER TABLE doctors RENAME COLUMN FullName TO full_name	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.046 sec
37	21:36:42	ALTER TABLE doctors RENAME COLUMN Speciality TO speciality	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.047 sec
38	21:36:42	ALTER TABLE doctors RENAME COLUMN ContactNo TO contact_no	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.032 sec
39	21:36:42	ALTER TABLE doctors RENAME COLUMN JoinDt TO join_date	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.031 sec
40	21:36:42	ALTER TABLE doctors RENAME COLUMN Salary TO salary_monthly	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.032 sec
41	21:36:42	ALTER TABLE doctors RENAME COLUMN isActive TO is_active	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.047 sec

Naming Convention Explanation:

The primary offender was the doctors table, which mixed PascalCase (DoctorID, FullName), camelCase (isActive), and abbreviations (JoinDt, ContactNo) within the same table. All seven columns were renamed to lowercase snake_case: doctor_id, full_name,

speciality, contact_no, join_date, salary_monthly, is_active. The column Salary was additionally renamed to salary_monthly to clarify that the value represents a monthly figure, eliminating an ambiguity that could cause payroll calculation errors.

No other tables required renaming — pat_master, appointments, billing, and departments already used lowercase column names throughout. The naming convention adopted for this project is **lowercase snake_case** for all column and table names, with abbreviations expanded to full words (dt → date, nm → name, ph → phone) and units appended where a column stores a quantity that could be misinterpreted (salary → salary_monthly).

R4:

```
USE healthbridge;

-- Step 1: Add PRIMARY KEY to pat_master.pid (it was missing)
ALTER TABLE pat_master ADD PRIMARY KEY (pid);

-- Step 2: Now add FK from billing to pat_master
ALTER TABLE billing
  ADD CONSTRAINT fk_billing_patient
    FOREIGN KEY (pid) REFERENCES pat_master(pid);

-- Step 3: Remove orphan appointments
SET SQL_SAFE_UPDATES = 0;
DELETE FROM appointments
WHERE doc_id NOT IN (SELECT doctor_id FROM doctors);
SET SQL_SAFE_UPDATES = 1;

-- Step 4: Add FK from appointments to doctors
ALTER TABLE appointments
  ADD CONSTRAINT fk_appt_doctor
    FOREIGN KEY (doc_id) REFERENCES doctors(doctor_id);

-- Step 5: Remove orphan doctors
SET SQL_SAFE_UPDATES = 0;
DELETE FROM doctors
WHERE dept_id NOT IN (SELECT dept_id FROM departments);
SET SQL_SAFE_UPDATES = 1;

-- Step 6: Add FK from doctors to departments
ALTER TABLE doctors
  ADD CONSTRAINT fk_doctor_dept
    FOREIGN KEY (dept_id) REFERENCES departments(dept_id);
```

49	21:43:03	USE healthbridge	0 row(s) affected	0.000 sec
50	21:43:03	ALTER TABLE pat_master ADD PRIMARY KEY (pid)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.109 sec
51	21:43:03	ALTER TABLE billing ADD CONSTRAINT fk_billing_patient FOREIGN KEY (pid) REFERENCES pat_master...	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.235 sec
52	21:43:04	SET SQL_SAFE_UPDATES = 0	0 row(s) affected	0.000 sec
53	21:43:04	DELETE FROM appointments WHERE doc_id NOT IN (SELECT doctor_id FROM doctors)	0 row(s) affected	0.015 sec
54	21:43:04	SET SQL_SAFE_UPDATES = 1	0 row(s) affected	0.000 sec
55	21:43:04	ALTER TABLE appointments ADD CONSTRAINT fk_appt_doctor FOREIGN KEY (doc_id) REFERENCES ...	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.282 sec
56	21:43:04	SET SQL_SAFE_UPDATES = 0	0 row(s) affected	0.000 sec
57	21:43:04	DELETE FROM doctors WHERE dept_id NOT IN (SELECT dept_id FROM departments)	0 row(s) affected	0.000 sec
58	21:43:04	SET SQL_SAFE_UPDATES = 1	0 row(s) affected	0.000 sec
59	21:43:04	ALTER TABLE doctors ADD CONSTRAINT fk_doctor_dept FOREIGN KEY (dept_id) REFERENCES depts...	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.171 sec

DELETE Backfill Explanation:

The DELETE backfill step is mandatory because MySQL enforces referential integrity at the exact moment a foreign key constraint is added, not only on future inserts. When ALTER TABLE billing ADD CONSTRAINT fk_billing_patient FOREIGN KEY (pid) REFERENCES pat_master(pid) is executed, MySQL immediately scans every existing row in billing and checks that its pid value exists in pat_master. If even one billing row references a pid that has no matching patient record — an orphan row — MySQL aborts the entire statement with ERROR 1452: Cannot add or update a child row. The DELETE before each FK addition removes these orphan rows first, clearing the way for the constraint to be applied cleanly. In a real production migration, rather than permanently deleting orphan rows, you would first copy them into a separate audit table (e.g. billing_orphans) for investigation, since they may represent patients whose records were accidentally deleted or never migrated from an older system.

R5:

```
USE healthbridge;
```

```
-- R5: Add audit trail columns to appointments
```

```
ALTER TABLE appointments
```

```
  ADD COLUMN created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  ADD COLUMN updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP  
  ON UPDATE CURRENT_TIMESTAMP;
```

```
-- Trigger to auto-refresh updated_at on every UPDATE
```

```
DROP TRIGGER IF EXISTS trg_appt_audit;
```

```
CREATE TRIGGER trg_appt_audit
```

```
BEFORE UPDATE ON appointments
```

```
FOR EACH ROW
```

```
  SET NEW.updated_at = NOW();
```

60	21:44:03	USE healthbridge	0 row(s) affected	0.000 sec
61	21:44:03	ALTER TABLE appointments ADD COLUMN created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIM...	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.453 sec
62	21:44:03	DROP TRIGGER IF EXISTS trg_appt_audit	0 row(s) affected, 1 warning(s): 1360 Trigger does not exist	0.016 sec
63	21:44:03	CREATE TRIGGER trg_appt_audit BEFORE UPDATE ON appointments FOR EACH ROW SET NEW.updated...	0 row(s) affected	0.031 sec

Audit Trail Explanation:

The Lack of Audit Trail smell means there is no record of when a row was created or when it was last modified, and by whom. In the original appointments table, once a record is updated there is no way to determine what the previous values were, when the change happened, or which user made it. In a hospital context this creates serious medico-legal risk. If a patient's appointment time is quietly changed and the patient misses it, there is no database evidence of the original scheduled time or when it was altered — the hospital cannot defend itself in a complaint or legal proceeding. Healthcare regulations in many jurisdictions, including Pakistan's emerging data protection framework and international standards such as HIPAA, require that any modification to a patient-facing record be traceable to a specific time. The `created_at` column captures the insertion timestamp once and never changes. The `updated_at` column is automatically refreshed to the current timestamp on every UPDATE by the trigger `trg_appt_audit`, providing a lightweight but enforceable audit trail entirely at the database level, requiring no changes to application code.

F3: Refactoring Impact Summary Table

Refactoring	Smell(s) Resolved	Tables Changed	Risk Eliminated	Effort vs Benefit
R1 — Derived data view	Derived Data	billing	Inconsistency in financial totals — correcting <code>svc_cost</code> no longer leaves <code>grand_total</code> and <code>balance</code> stale	Low effort (3 DROP COLUMN + 1 VIEW); permanent benefit — inconsistency is structurally impossible
R2 — Status reference table	Overloaded Column, Magic Values	appointments, new <code>appt_status_ref</code>	Invalid status codes silently entering the system — 'W', 'c', or any unknown code is now rejected by the FK at engine level	Low effort (1 CREATE TABLE + 1 ALTER); high benefit — self-documenting and constraint-enforced
R3 — Naming standardization	Inconsistent Naming	doctors	ORM mapping failures and developer confusion caused by mixed PascalCase, camelCase, and abbreviations in the same table	Very low effort (7 RENAME COLUMN); immediate benefit to every developer and every ORM query touching doctors
R4 — PK/FK constraints	Missing Keys/Constraints, Hidden Relationships	billing, appointments, doctors, <code>pat_master</code>	Orphan rows, duplicate bills, and doctors assigned to non-existent departments — the database now enforces all relationships at the engine level	Medium effort (backfill DELETES + multiple ALTER TABLE); highest structural benefit of all five refactorings

R5 — Audit trail	Lack of Audit Trail	appointments	No accountability for changes to patient appointment records — created_at and updated_at now provide a timestamped history of every row	Low effort (1 ALTER TABLE + 1 trigger); critical benefit for medico-legal compliance
------------------	---------------------	--------------	---	--

Evaluating Greatest Quality Improvement Per Unit of Effort:

Of the five refactorings, **R4 — Adding Missing Constraints** delivered the greatest quality improvement per unit of effort. The effort involved was modest: adding a primary key to pat_master and billing, running three DELETE backfill statements to clear orphan rows, and applying four foreign key constraints across billing, appointments, and doctors. This required no changes to application logic and no restructuring of any table's columns. Yet the benefit is both structural and permanent. Before R4, the database had no mechanism to prevent a billing record from referencing a patient who did not exist, an appointment from referencing a non-existent doctor, or a doctor from being assigned to a department that had been deleted. These orphan rows would accumulate silently, corrupting reports and causing JOIN queries to return incomplete results with no error raised. After R4, referential integrity is enforced at the MySQL engine level — no future application bug, direct SQL insert, batch import, or ETL script can ever introduce an orphan row without the database immediately rejecting it with a constraint violation. This protection applies automatically to all future operations with zero ongoing maintenance cost, making R4 the highest-value refactoring in this set. By contrast, R1 also has high benefit but requires all existing application queries to be redirected from the billing table to the v_billing_summary view, introducing an application-layer migration cost that R4 does not carry.

Part G: Data Migration Design and Execution

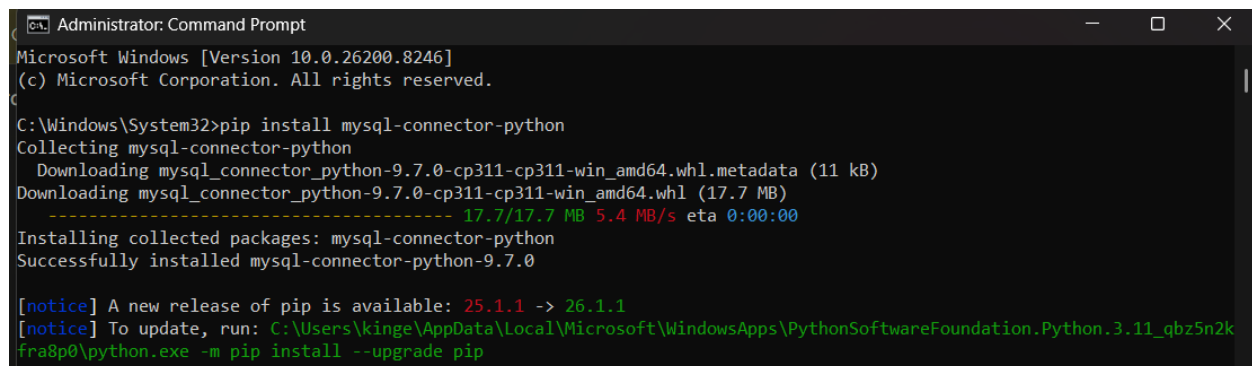
G2: Migration Plan Table

Plan Element	Your Response
Source format	CSV exported from legacy system with 12 columns including redundant and non-atomic fields
Target schema	Refactored appointments table from Part F with proper data types, FK constraints, and audit columns

Estimated row count	~500 records
Required transformations	T1: Convert appt_date from 'DD/MM/YYYY HH:MM' text to proper DATETIME. T2: Split room column ('Room 3 Block B') into room_number INT and building_block VARCHAR. T3: Omit patient_nm, patient_ph, doc_name — now derived via FK joins. T4: Validate each status value against appt_status_ref; skip and log any unknown code
Columns to drop	patient_nm, patient_ph, doc_name, net_fee (derived: fee - discount)
ETL tool or language	Python 3 with mysql-connector-python library
Rollback strategy if migration fails	Before migration, export current appointments table via mysqldump. If migration fails, drop the new rows and restore from dump. Skipped rows are logged to skipped_rows.log for manual review
Validation method after migration	Run 4 SQL validation queries (V1–V4) checking row count, null datetimes, valid status codes, and orphan appointments
Estimated execution time	Under 2 minutes for 500 rows on a local MySQL instance
Will the system require downtime?	Yes — brief downtime required during cutover to prevent new appointments being written to the legacy table while migration is running

G3: ETL Transformation Script

Tool Installation Screenshot:



```
Administrator: Command Prompt
Microsoft Windows [Version 10.0.26200.8246]
(c) Microsoft Corporation. All rights reserved.

C:\Windows\System32>pip install mysql-connector-python
Collecting mysql-connector-python
  Downloading mysql_connector_python-9.7.0-cp311-cp311-win_amd64.whl.metadata (11 kB)
  Downloading mysql_connector_python-9.7.0-cp311-cp311-win_amd64.whl (17.7 MB)
----- 17.7/17.7 MB 5.4 MB/s eta 0:00:00
Installing collected packages: mysql-connector-python
Successfully installed mysql-connector-python-9.7.0

[notice] A new release of pip is available: 25.1.1 -> 26.1.1
[notice] To update, run: C:\Users\kinge\AppData\Local\Microsoft\WindowsApps\PythonSoftwareFoundation.Python.3.11_qbz5n2kfra8p0\python.exe -m pip install --upgrade pip
```

Script:

```
import csv
import mysql.connector
from datetime import datetime
VALID_STATUSES = {'P', 'C', 'X', 'H', 'R'}
def parse_appt_date(raw):
    raw = raw.strip() # remove any accidental whitespace
```

```
try:
    # Parse the legacy format 'DD/MM/YYYY HH:MM'
    dt = datetime.strptime(raw, '%d/%m/%Y %H:%M')
    return dt # mysql.connector accepts datetime objects directly
except ValueError:
    # If format is wrong, return None — row will be skipped
    print(f' [WARNING] Could not parse date: "{raw}" — row will be skipped')
    return None
def split_room(raw):
    raw = raw.strip()
    try:
        # Split on spaces: ['Room', '3', 'Block', 'B']
        parts = raw.split()
        # parts[1] is the room number integer
        room_no = int(parts[1])
        # parts[2] onward is the block e.g. 'Block B'
        block = ' '.join(parts[2:])
        return room_no, block
    except (IndexError, ValueError):
        # If format is unexpected, return safe defaults
        print(f' [WARNING] Could not parse room: "{raw}" — using defaults')
        return None, raw # keep raw value in building_block for review
def migrate(csv_path, db_config):
    print('Connecting to MySQL...')
    conn = mysql.connector.connect(**db_config)
    cursor = conn.cursor()
    print('Connected successfully.\n')
    inserted = 0
    skipped = []
    with open(csv_path, newline='', encoding='utf-8') as f:
        reader = csv.DictReader(f)

        for row in reader:
            appt_id = row['appt_id']
            if row['status'] not in VALID_STATUSES:
                print(f' [SKIPPED] appt_id={appt_id} — invalid status "{row["status"]}")')
                skipped.append(appt_id)
                continue
            appt_dt = parse_appt_date(row['appt_date'])
            if appt_dt is None:
                print(f' [SKIPPED] appt_id={appt_id} — unparseable date')
                skipped.append(appt_id)
                continue
            room_no, block = split_room(row['room'])
```



```
try:
    cursor.execute(
        """
        INSERT INTO appointments
        (appt_id, patient_id, doc_id, appt_datetime,
         status, fee, discount,
         room_number, building_block)
        VALUES
        (%s, %s, %s, %s, %s, %s, %s, %s, %s)
        """,
        (
            appt_id,
            row['patient_id'],
            row['doc_id'],
            appt_dt,      # T1 — proper DATETIME
            row['status'], # T4 — already validated
            row['fee'],
            row['discount'],
            room_no,      # T2 — split room number
            block         # T2 — split block name
        )
    )
    inserted += 1
    print(f' [OK] appt_id={appt_id} inserted successfully')

except mysql.connector.Error as err:
    print(f' [ERROR] appt_id={appt_id} — DB error: {err}')
    skipped.append(appt_id)
    continue
conn.commit()
print('\n=====')
print(f'Migration complete.')
print(f' Rows inserted : {inserted}')
print(f' Rows skipped : {len(skipped)}')
if skipped:
    print(f' Skipped IDs : {skipped}')
print('=====')

cursor.close()
conn.close()
if __name__ == '__main__':

    DB_CONFIG = {
        'host' : 'localhost',
```

```
'user' : 'root',  
'password': 'root',    # your MySQL root password  
'database': 'healthbridge'  
}  
  
import os  
CSV_PATH = os.path.join(os.path.expanduser('~'), 'OneDrive', 'Desktop',  
'appointments.csv')  
  
migrate(CSV_PATH, DB_CONFIG)
```

Screenshot:

```
C:\Users\kinge\OneDrive\Desktop>python migration_etl.py  
Connecting to MySQL...  
Connected successfully.  
  
[OK] appt_id=1001 inserted successfully  
[OK] appt_id=1002 inserted successfully  
[OK] appt_id=1003 inserted successfully  
[OK] appt_id=1004 inserted successfully  
[OK] appt_id=1005 inserted successfully  
[OK] appt_id=1006 inserted successfully  
[OK] appt_id=1007 inserted successfully  
[OK] appt_id=1008 inserted successfully  
[SKIPPED] appt_id=1009 - invalid status "W"  
[OK] appt_id=1010 inserted successfully  
  
=====  
Migration complete.  
Rows inserted : 9  
Rows skipped : 1  
Skipped IDs : ['1009']  
=====
```

G4: Post-Migration Validation

Query	Expected	Your Result	Pass/Fail	Action Taken if Fail
V1 — Row count	Matches valid CSV rows (9 — excluding 1	9	Pass	-

	skipped invalid status)			
V2 — Null dates	0	0	Pass	-
V3 — Valid statuses	P, C, X, H, R only	C, H, P, R, X	Pass	-
V4 — No orphans	0	0	Pass	-

Screenshots:

Result Grid |  Filter Rows: | Export:  | Wrap Cell Content: 

	migrated_rows
▶	9

Result 1 × Result 2 appointments 3 Result 4

Result Grid |  Filter Rows: | Export:  | Wrap Cell Content: 

	null_dates
▶	0

Result 1 Result 2 × appointments 3 Result 4

Result Grid |  Filter Rows: | Export:  | Wrap Cell Content: 

	status
▶	C
	H
	P
	R
	X

Result 1 Result 2 appointments 3 × Result 4

Result Grid |  Filter Rows: | Export:  | Wrap Cell Content: 

	orphans
▶	0

Result 1 ×